Making sense of language models with structured content plans

Fei-Tzin Lee

Columbia University

September 12, 2024

What do we want from our language-processing machines?

We want to understand the meaning of existing language.

We want to produce language that makes sense – we want it to be semantically consistent with itself, and possibly with external knowledge

What's the problem?

Language models don't do this naturally!

- Intrinsic and extrinsic hallucinations
- Factual errors
- Nonsensical content

How do we address this?

We need a more principled way of interacting with content.

How do we address this?

We propose to supplement language models by making meaning explicit with a human-interpretable representation.

Our hypothesis: the interactivity this representation gives us is its most useful – and still underutilized – feature.

How do we represent meaning?

We choose to use Abstract Meaning Representation (AMR):

- Human-interpretable, but usable by machines
- Relatively lossless representation of sentence-level semantics
- Well-studied; active work in both parsing and generation

The AMR-text cycle

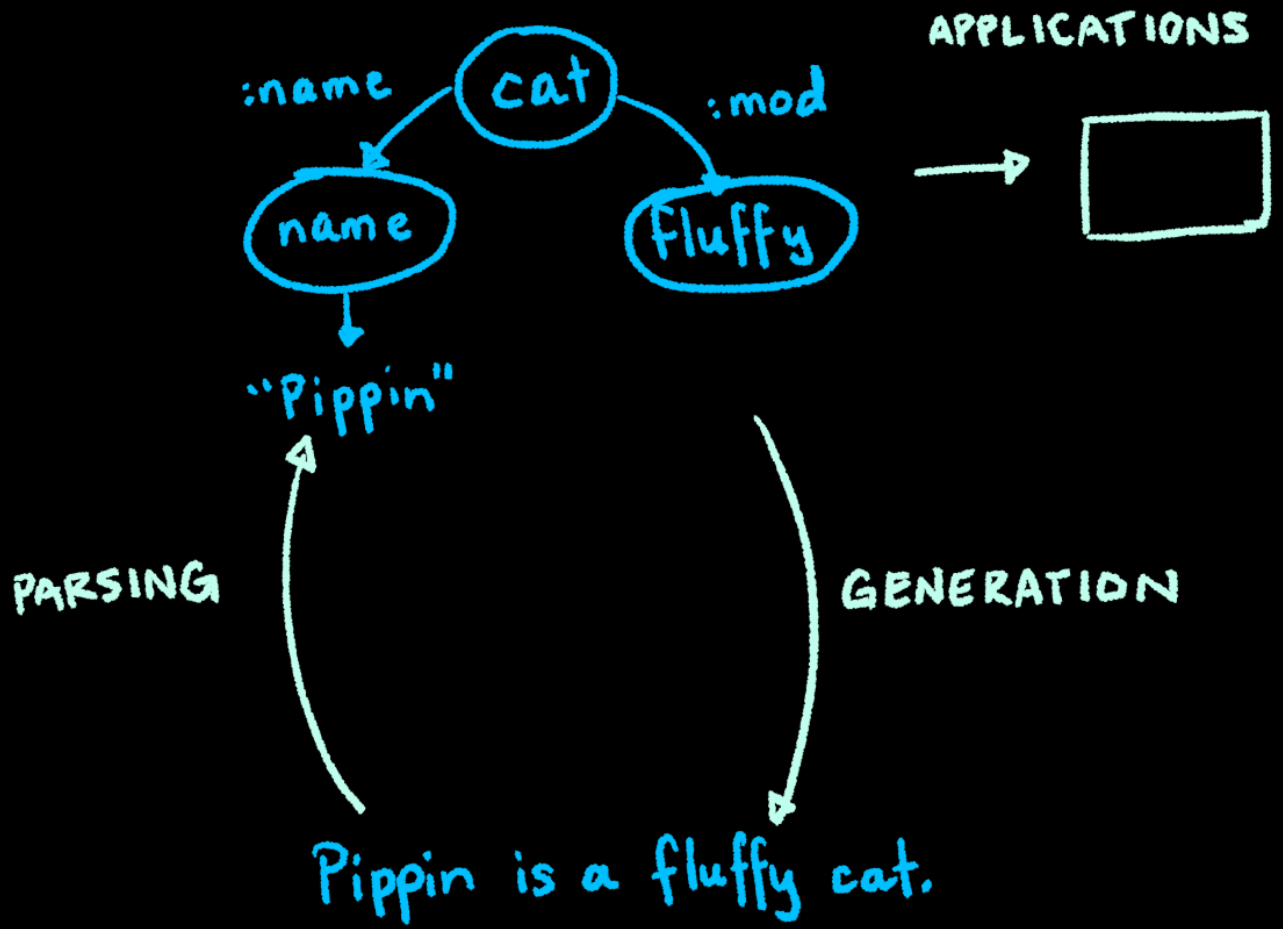


Figure: Tasks in AMR

Challenges

AMR is useful, but underutilized. We address three previously unexplored areas of prior work:

- Limitations on context
- Using AMR for content analysis
- Using AMR as an interactive content plan

In addressing these, we demonstrate that AMR can provide new and complementary ways of interacting with the meaning of text.

Contributions

In addressing limitations on context, we:

1. Create a novel method of AMR graph merging which yields higher-quality document-level representations;
2. Conduct the first evaluation of node merging on downstream content selection performance;
3. Collect the first hand-annotated dataset of inter-sentence node alignments between document-summary AMR pairs.

Contributions

In using AMR for content analysis, we:

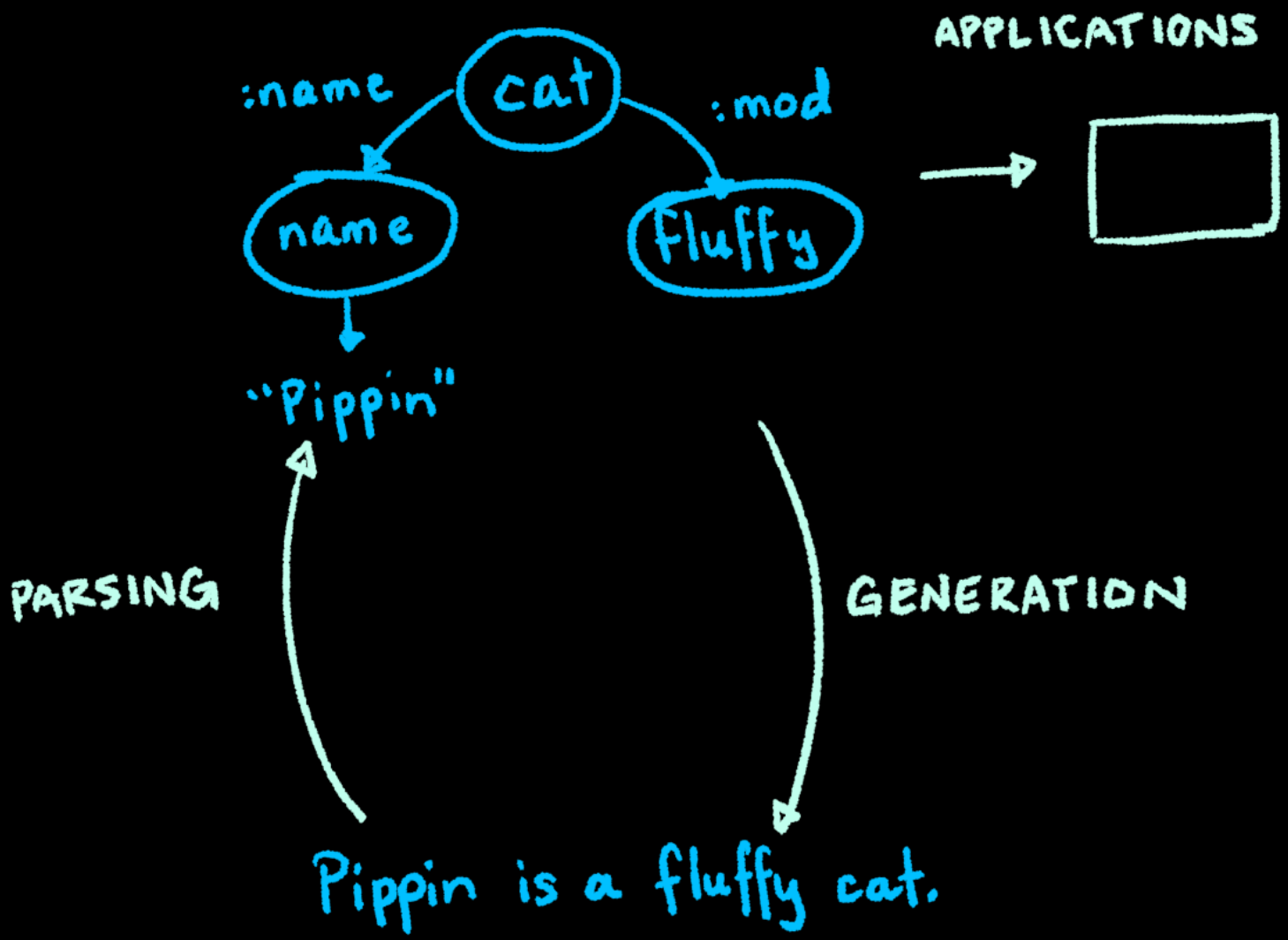
1. Create a set of novel approaches to node-level AMR alignment using graph neural networks, and demonstrate enormous improvements ($\approx 2\times$) over baseline;
2. Propose a novel family of AMR-based text evaluation metrics, and find that they have complementary strengths relative to existing metrics.

Contributions

In using AMR as a content plan for generation, we:

1. Propose a novel methodology for using AMR as a content plan for fine-grained (syntactic and semantic) control over generated text;
2. Demonstrate that AMR yields reliable and consistent control over concept-level aspects of generated text.

Overview


APPLICATIONS
:name
cat
:mod
name
Fluffy
"Pippin"
PARSING
GENERATION
Pippin is a fluffy cat.

An introduction to AMR

AMR is a graphical semantic representation which represents concepts as nodes and relations between those concepts as edges.

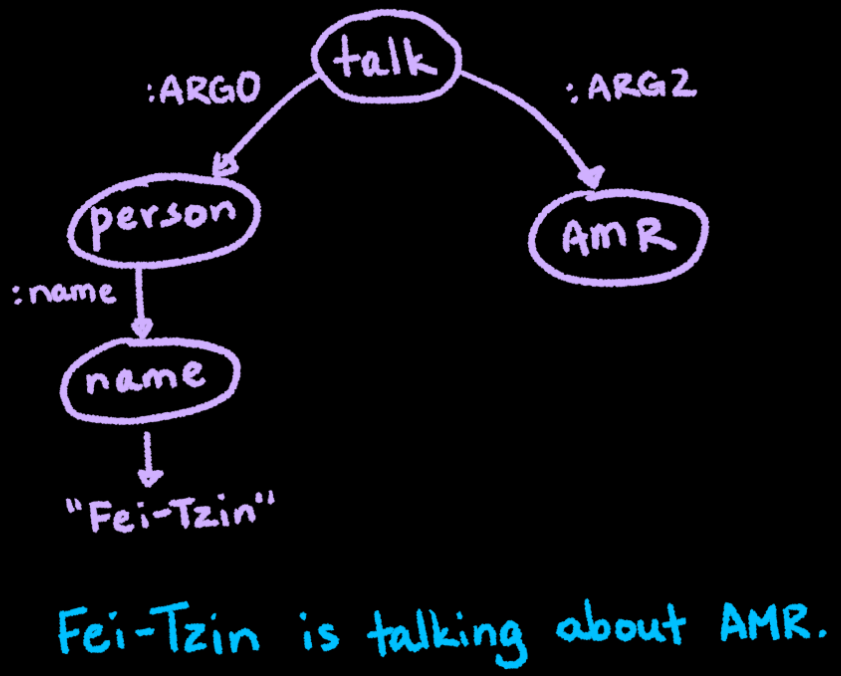


Figure: Simple example AMR.

An introduction to AMR

In its basic form, AMR is a sentence-level semantic representation.

The canonical representation for a passage of several sentences is simply a collection of disjoint AMRs with no connection between them.

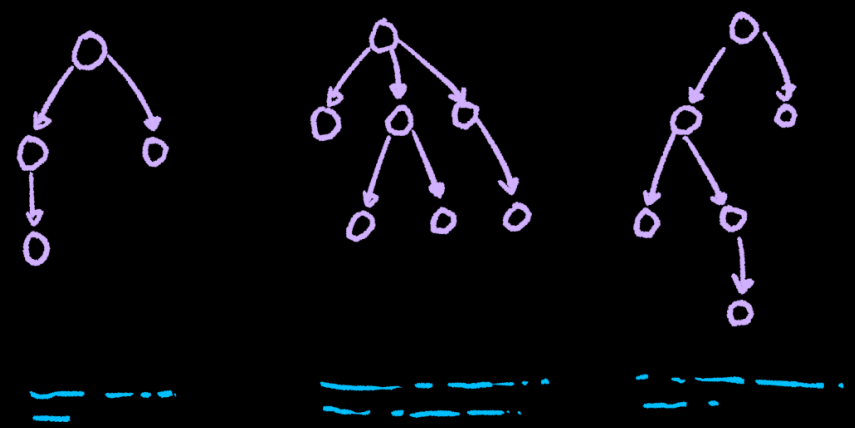


Figure: AMR representation for a multi-sentence passage.

Why is this not enough?

There are many tasks for which sentence-level context is sufficient (paraphrase detection, entailment, sentiment analysis).

There are also many tasks that require understanding longer texts: summarization, QA, dialogue, translation, and so on all require longer-range understanding of context.

Application

Our target application is summarization.

The AMR summarization pipeline involves a few steps:

1. Graph construction;
2. Graph-based content selection;
3. Summary generation from content graph

A unified representation for longer contexts

Ideally, we don't want to rely on our models to guess at how to put together a collection of independent AMRs.

Our aim is to create a single document-level semantic representation from AMRs.

Document-level AMR

We merge nodes across sentences to create a document-level graph.

Claim: we want one node per entity, no matter how many times it appears in the text.

Prior approaches

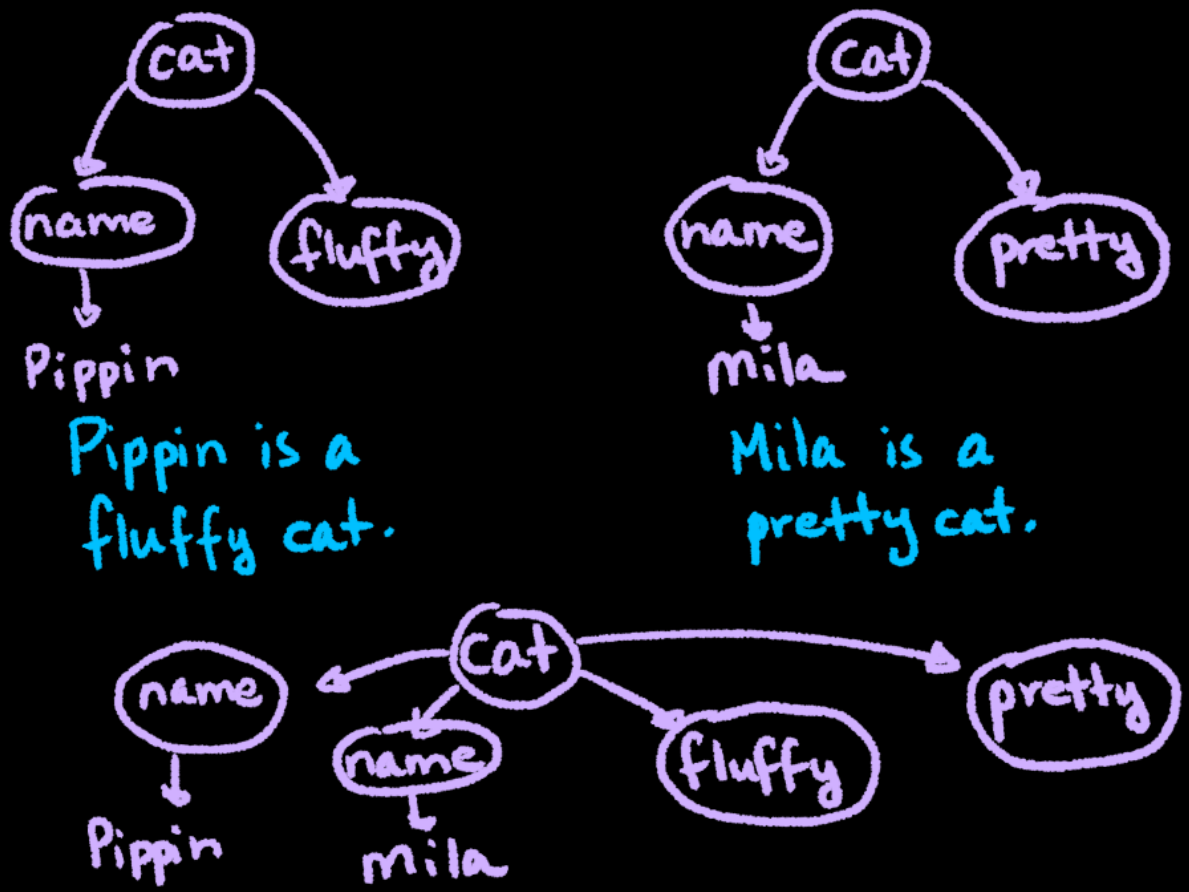
At the time we approached this problem, two lines of work on long-context AMR construction existed:

- Document graph construction for summarization;
- Coreference for AMR.

We focus on the former, as the latter creates a different kind of representation.

Prior work

The prior approach was simple: merge all nodes with identical concept labels.



Undermerging in prior work

Error type	Example sentence fragment pair that produces error
Missing synonyms	...nuclear bombs... ...nuclear arms...
Name skipping	Estonian informatics center spokeswoman Katrin pargmae stated... Katrin pargmae also stated...

Table: Missed merges from concept merging.

Overmerging in prior work

Error type	Example sentence fragment pair that produces error
Stopword conflation	...Swedish law and order... Both Norway and Sweden...
Concept conflation (surface)	...any person whose assets are being frozen... ...decision to freeze the assets of...
Concept conflation (hidden)	...the foreign minister of India... ...the foreign minister of China...

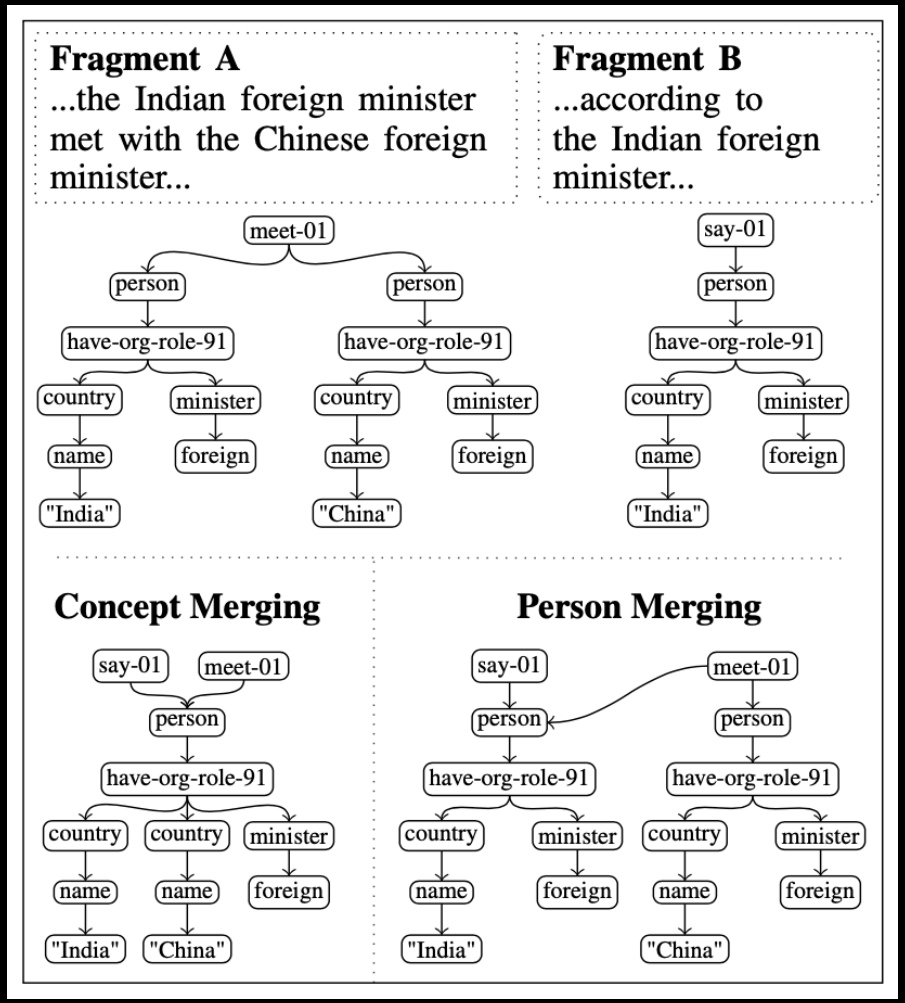
Table: Incorrect merges from concept merging.

Our approach

Based on our manual analysis of errors from concept-based merging, we introduce two new methods of node merging:

- Person-based merging, where we only merge person nodes and their descendants which are inside coreferent text spans;
- Combined merging, where we first perform person merging, then use concept merging on remaining nodes.

Prior work versus our approach



Evaluation methodology

We are interested in two kinds of evaluation:

- Intrinsic: cluster label correctness of node merges
- Extrinsic: downstream performance on content selection

Data collection

We collect a dataset of human annotations over document and summary AMR nodes from the proxy subsection of the AMR Bank.

- Objective: match document and summary nodes that refer to same concept
- Intrinsic: nodes refer to same summary node
- Extrinsic: nodes refer to any summary node

Results

	B^3			LEA		
Method	Prec.	Recall	F1	Prec.	Recall	F1
Person	0.011	0.023	0.015	0.052	0.144	0.074
Concept*†	0.469	0.514	0.490	**0.088**	0.191	0.115
Combined*†	**0.743**	**0.704**	**0.723**	0.086	**0.241**	**0.122**

- Person-based merging does quite poorly on its own
- However, person merging is a useful filter for combined merging – it eliminates things we don't want to merge

Results

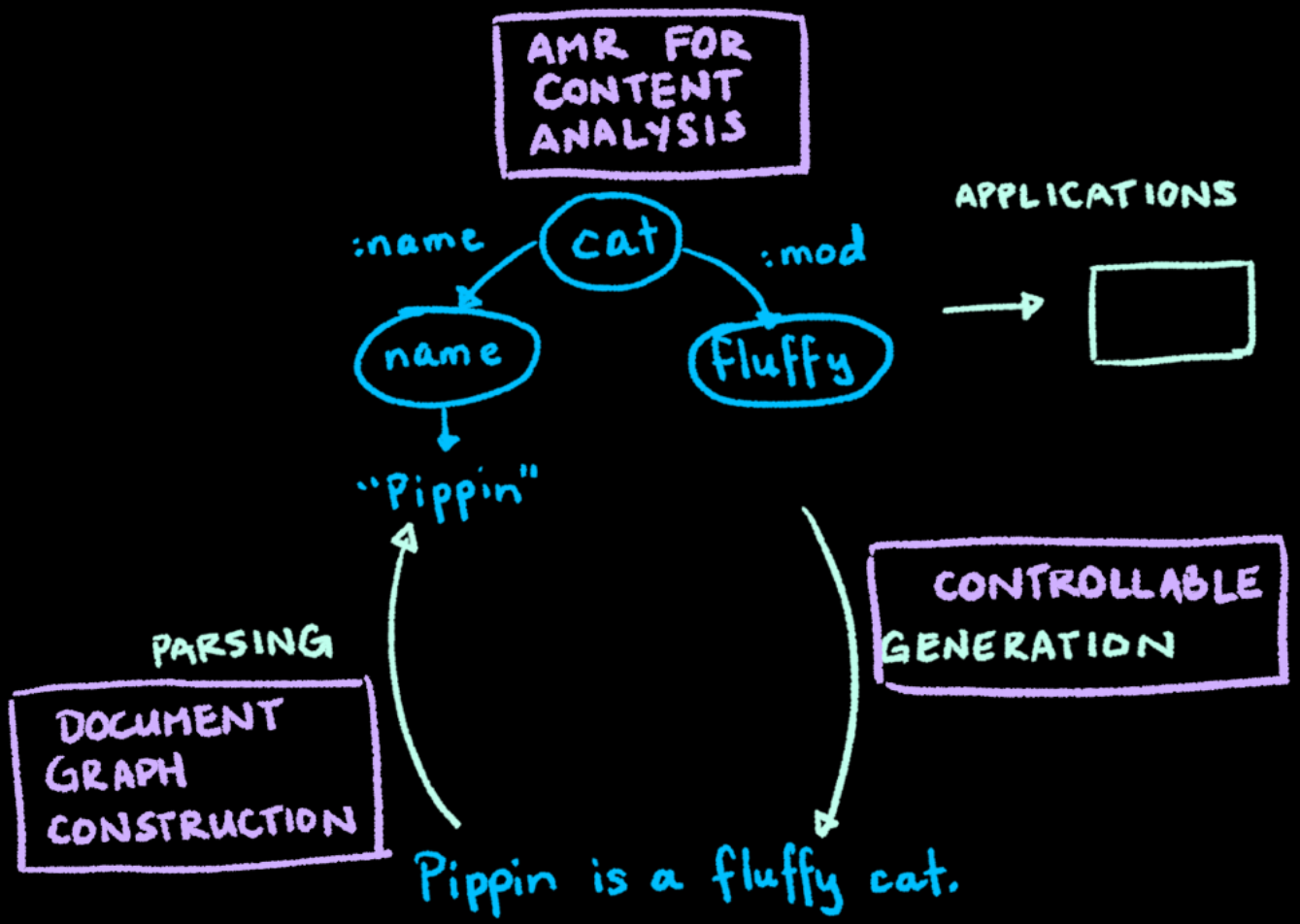
Merge Strategy	Prec.	Recall	F1
Person	0.348	0.244	0.280
Concept (Liu et al.)*	0.363	0.277	0.293
Unmerged*	0.327	0.359	0.332
Combined*	**0.412**	**0.400**	**0.398**

- Even unmerged graphs are surprisingly acceptable input
- Good merging is still useful

Conclusions

- A good input representation is crucial for downstream performance
- Analyzing error cases from prior work allows us to create better ways of representing long context with AMR

Overview


AMR FOR CONTENT ANALYSIS
cat
:name
:mod
name
Fluffy
"Pippin"
APPLICATIONS
CONTROLLABLE GENERATION
PARSING
DOCUMENT GRAPH CONSTRUCTION
Pippin is a fluffy cat.

Using AMR to its fullest potential

There are well-studied tasks in the AMR literature: parsing text into AMR, AMR-based applications, generating text from AMR.

Applications usually use AMR as fixed input to a downstream model. In contrast, we are interested in what AMR can tell us about content.

A missing step?

Rather than applying parsed AMR directly as input to another task, what can we do with the AMR itself?

- Find similar content with AMR
- Evaluate generated text on semantics with AMR

These are natural applications of AMR's most useful quality: abstraction from surface noise.

AMR alignment

First problem: using AMR to align text from disparate documents.

An obvious application for this is, again, summarization, but this can be used to analyze generated text in other ways.

Methodology

Goal: use a graph-based neural network to learn alignment-friendly node embeddings.

Experiments:

- Initial embedding
- Model architecture
- Pretraining tasks

Inferring alignment from embeddings

Approach:

1. Learn an embedding for each node;
2. Use a threshold over cosine similarity to decide alignment.

We finally use our alignment dataset for its direct purpose.

Results

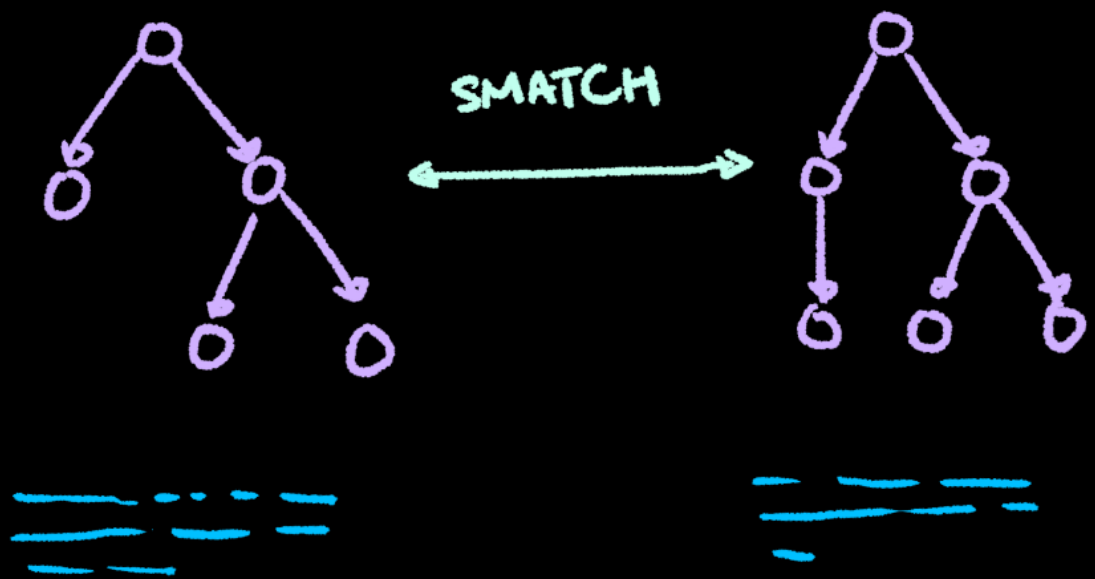
Alignment approach	Precision	Recall	F1
Exact match	0.31	0.86	0.45
Word2Vec	0.27	0.87	0.41
BERT	0.41	0.57	0.48
GAT with BERT base and pre-training	0.64	0.67	0.70
w2v + BERT	0.71	0.64	0.67
w2v + contextual embedding ensemble	0.84	0.72	0.77
w2v + contextual + GAT ensemble	**0.91**	**0.78**	**0.84**

- Context in representations is useful, but static word embeddings are even less useful than plain text-based exact match
- Graph context is useful in a different and additive way over text-based context
- Representations matter! Ensembling gets us a large boost in performance.

AMR for evaluation

Our second problem involves using AMR for evaluation of model-generated text.

Approach: parse generated text, reference into graphs; use an AMR comparison metric to directly compare graph structures.



Evaluation

To evaluate our metrics, we actually need to perform a meta-evaluation – how well does our metric correspond with human judgement?

We do this on two domains, data-to-text generation and summarization.

Meta-evaluation

Meta-evaluation: how strongly is our metric correlated with human judgement (on fluency, factuality, etc.)?

- Use a meta-evaluation dataset consisting of paired document/output/target texts with human ratings
- Compute correlations at sample level and system level

Sample level results

		WebNLG					SummEval			
Representation	Metric	Corr.	Cov.	Flu.	Rel.	Struct.	Coher.	Consist.	Flu.	Rel.
Text	ROUGE-1	**0.354**	**0.320**	0.311	**0.323**	0.293	0.184	**0.177**	0.133	**0.320**
	ROUGE-2	0.302	0.248	0.293	0.271	0.277	0.142	0.144	0.106	0.239
	ROUGE-L	0.272	0.227	0.276	0.240	0.256	**0.185**	0.158	0.118	0.267
Dependency	Smatch	0.302	0.253	**0.345**	0.258	**0.313**	0.117	0.144	**0.155**	0.256
	SemBleu	0.211	0.186	0.251	0.184	0.231	0.058	0.075	0.104	0.119
AMR	Smatch	0.333	0.302	0.300	0.292	0.277	0.128	0.075	0.067	0.220
	SemBleu	0.318	0.289	0.267	0.268	0.245	0.153	0.051	0.044	0.217

- Dependency parses tell us useful information about structure
- AMR doesn't give us strong sample-level benefit over text, but also doesn't fall too far behind

System level results

		WebNLG					SummEval			
Representation	Metric	Corr.	Cov.	Flu.	Rel.	Struct.	Coher.	Consist.	Flu.	Rel.
Text	ROUGE-1	0.695	0.718	0.851	0.802	0.856	0.309	0.610	0.565	0.623
	ROUGE-2	0.745	0.630	0.910	0.744	0.915	0.286	**0.632**	**0.595**	**0.663**
	ROUGE-L	**0.802**	0.694	0.874	0.807	0.871	**0.385**	0.527	0.537	0.650
Dependency	Smatch	0.448	0.588	**0.931**	0.657	**0.948**	0.059	0.463	0.428	0.518
	SemBleu	0.410	0.465	0.884	0.509	0.897	0.044	0.363	0.342	0.501
AMR	Smatch	0.600	**0.825**	0.898	**0.869**	0.888	0.132	0.343	0.337	0.537
	SemBleu	0.619	0.815	0.872	0.861	0.862	0.221	0.293	0.310	0.585

- Dependency and AMR parses shine in the system-level setting
- Strengths roughly follow our intuition about what each graph tells us

Conclusions

- Input representations matter
- Structure from semantic graphs tells us meaningfully different things than plain text
- Structured representations allow us to analyze text in ways that match our intuition

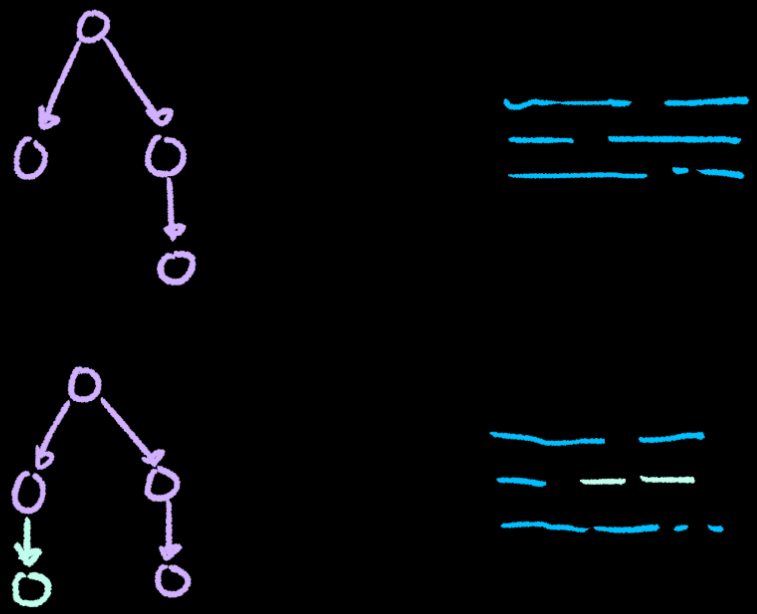
AMR for generation

AMR-to-text generation is an actively researched task. We continuously develop data, training methods, and architectures to generate better text.

What do we mean by better text?

Putting the planning in content plan

We want our content plans to be interactive and modular: we can modify the content plan and see our output change in predictable ways.



AMR as a scaffold for controlled generation

Goal: modify input AMR and see if the text follows.

This gives us the ability to control output at a fine-grained level.

Controlling syntax with an AMR skeleton

We start with syntax, which is not naturally represented in AMR.
Approach:

- Linearize the AMR into a sequential format;
- Add syntactic tags to the AMR;
- Finetune an existing AMR-to-text model to use tagged AMR

Inserting and validating syntactic tags

To gather syntactic tags from input, we deterministically find syntactic labels based on dependency parse annotations.

We consider three kinds of syntax: verb voice, verb tense, and entity surface realization.

Methodology

We finetune AMRBART on (1) plain AMR-to-text generation, as a baseline, and (2) tagged AMR to text, with multiple varieties of tagged AMR.

To evaluate, we automatically infer syntactic tags from the generated output, and compare to the expected tags.

Results

	Test set		Proxy test set	
Model	F1	Flip	F1	Flip
Untagged	0.833	0.048	0.630	0.086
Voice	0.965	0.498	0.909	0.516
v + t	0.957	**0.500**	0.934	0.546
v + e	**0.972**	0.463	0.941	0.541
v + t + e	0.965	0.499	**0.979**	**0.581**

Table 6.5: F1 of finetuned BART variants for verb voice, in original and flipped settings, on our test set and the proxy test set. Components of combined models are abbreviated: voice (v), tense (t), entity (e).

- It works!
- We have substantial control even in the flipped setting. (Note performance cap due to active/passive limitations)

Results

Model	Flipping	Macro F1	Proxy F1	VB	VBD	VBG	VBN	VBP	VBZ
Untagged	None	0.691	0.448	0.830	0.747	0.687	0.757	0.562	0.564
Tense	None	**0.979**	**0.961**	**0.990**	**0.981**	**0.994**	**0.979**	0.949	0.982
v + t	None	0.978	0.953	0.988	0.976	0.992	0.974	**0.953**	**0.986**
v + t + e	None	0.971	0.941	0.986	0.968	0.991	0.967	0.933	0.983
Untagged	Flipped	0.185	0.196	0.826	0.114	0.035	0.075	0.006	0.055
Tense	Flipped	0.784	0.806	**0.986**	**0.909**	**0.871**	**0.830**	0.143	0.964
v + t	Flipped	**0.786**	**0.899**	0.983	0.902	0.859	0.800	**0.207**	**0.966**
v + t + e	Flipped	0.760	0.736	0.981	0.891	0.818	0.736	0.173	0.959

Table 6.6: Performance of finetuned BART models for verb tense. F1 is reported on both our test set and the proxy test set; individual class F1 scores are on our test set.

- Verb tense is even easier to learn than verb voice

Results

Model	Flipping	Name F1	Pronoun F1	she	he	they
Untagged	None	0.399	0.720	0.473	0.782	0.906
Entity	None	**0.407**	0.996	0.993	**0.998**	**0.998**
v + e	None	0.395	0.995	0.992	0.997	0.996
v + t + e	None	0.403	**0.999**	**1.000**	**0.998**	**0.998**
Untagged	Flipped	0.401	0.204	0.083	0.283	0.245
Entity	Flipped	0.399	**0.980**	**0.986**	0.985	**0.970**
v + e	Flipped	0.426	0.976	0.978	**0.988**	0.961
v + t + e	Flipped	**0.433**	0.967	0.974	0.975	0.953

Table 6.7: Performance of finetuned BART models for entity realization. F1 is reported for the binary named - not named task as well as for the pronoun generation task. Numbers here are only on our test set, as there were only 16 sentences remaining in the proxy set after filtering.

- Pronouns are easy, names are hard

Realizing entities

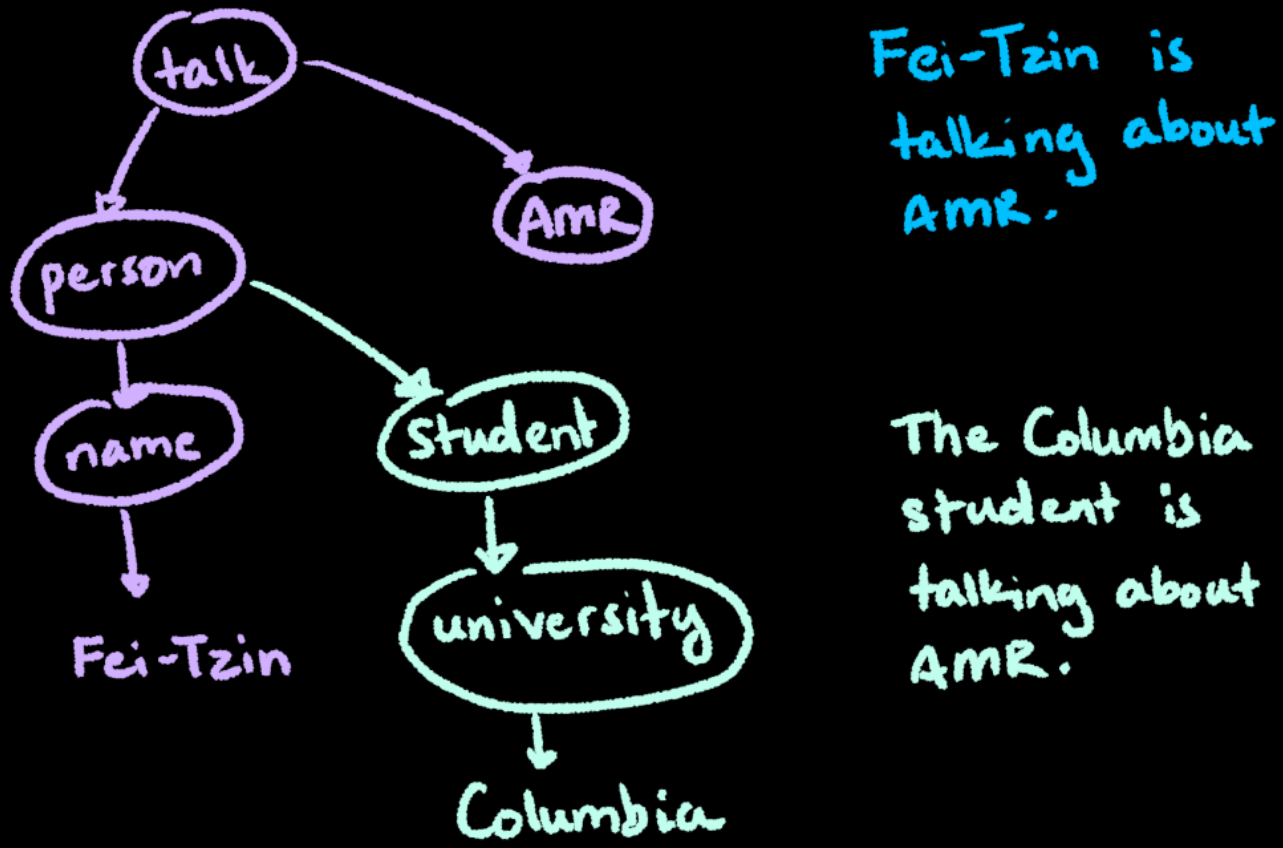


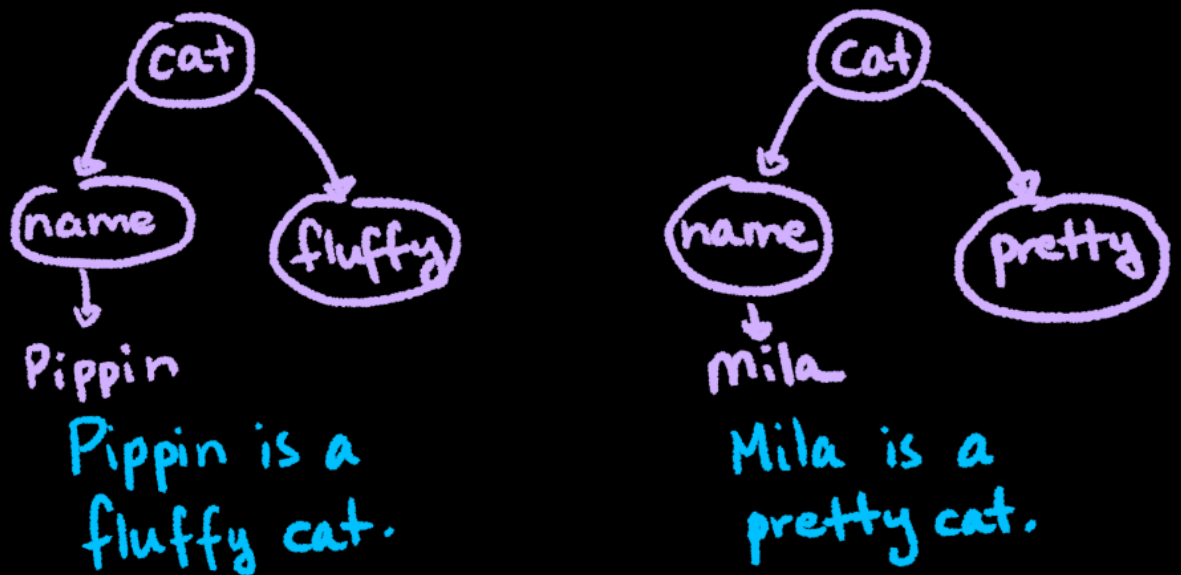
Figure: Two ways of realizing the same entity.

Looking forward: abstractive operations in text

We perform an initial exploration into two types of semantic abstraction:

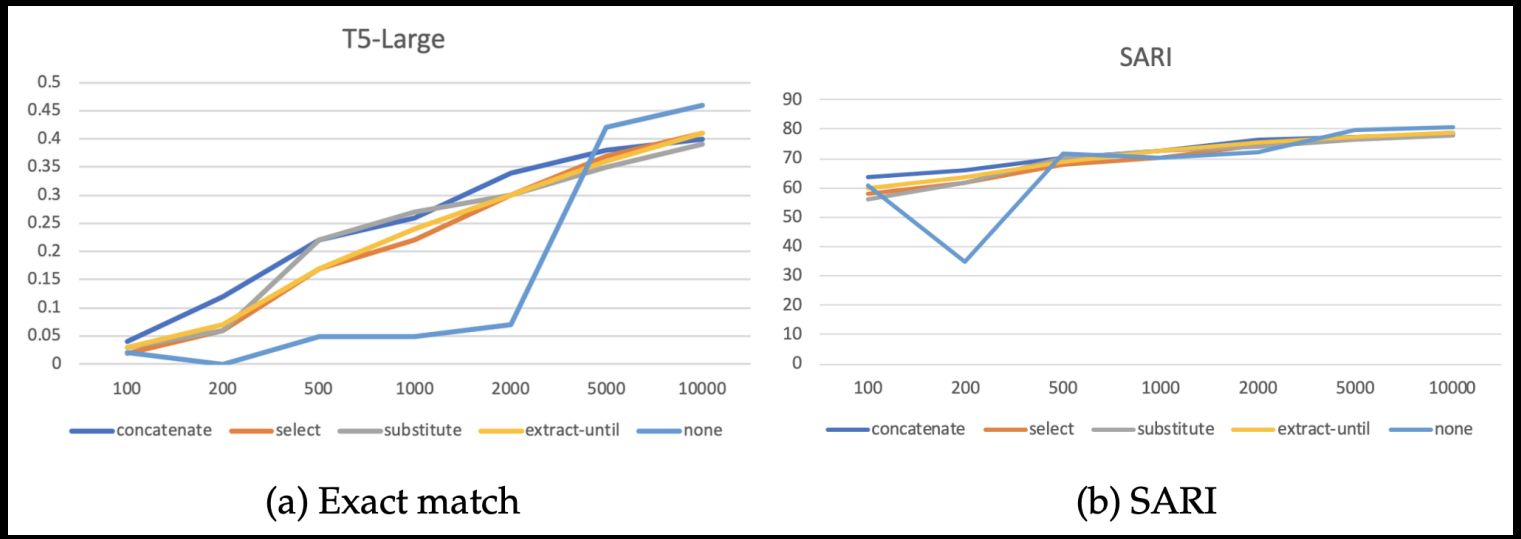
- Sentence fusion
- Concept aggregation

Sentence Fusion



Pippin is a fluffy cat, and Mila is a pretty cat.

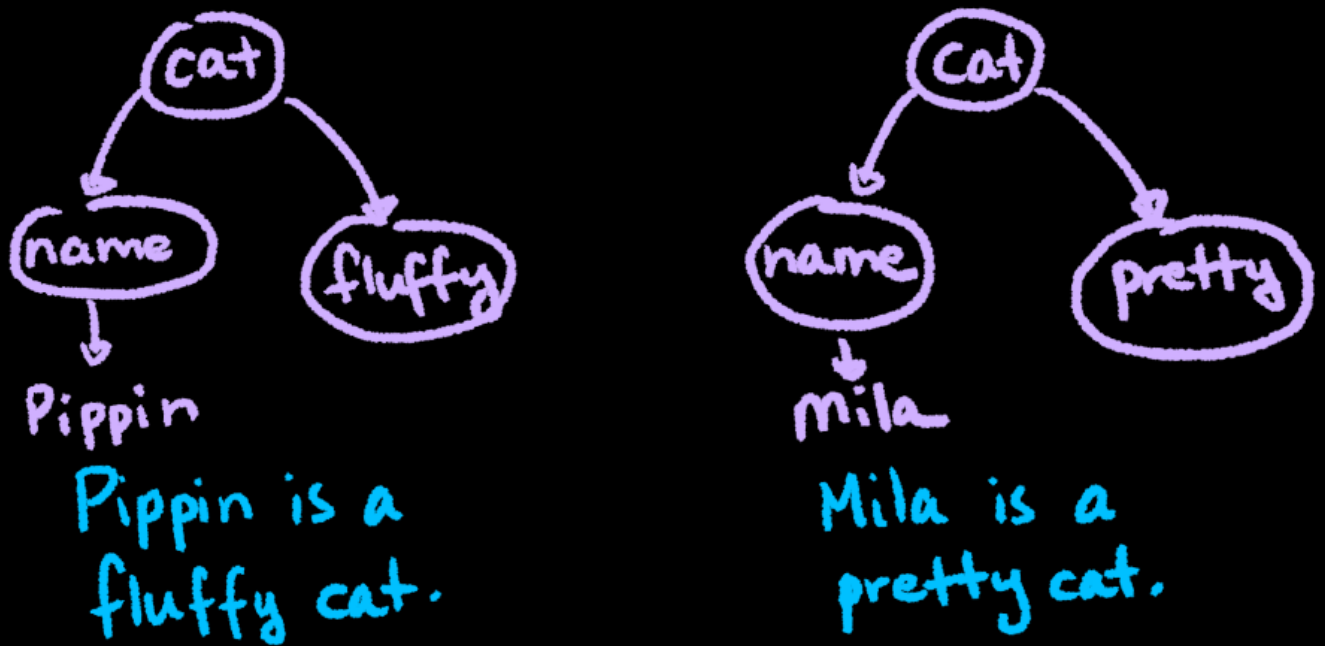
Pretraining for sentence fusion



(a) Exact match (b) SARI

- Pretraining helps a lot at small amounts of finetuning data
- Exact match is a less discriminating metric, but more discriminative in this case

Concept Aggregation



Dataset

We gather aggregation data by automatically finding lists of aggregatable concepts from news documents:

- Find lists of items separated by “and” or “or”
- Replace automatically in text

Aggregation

Model	SARI	Exact
AMR-BART (finetuned)	0.928	0.3

- There's a long way to go!
- Exact match is low, but this is a difficult task!

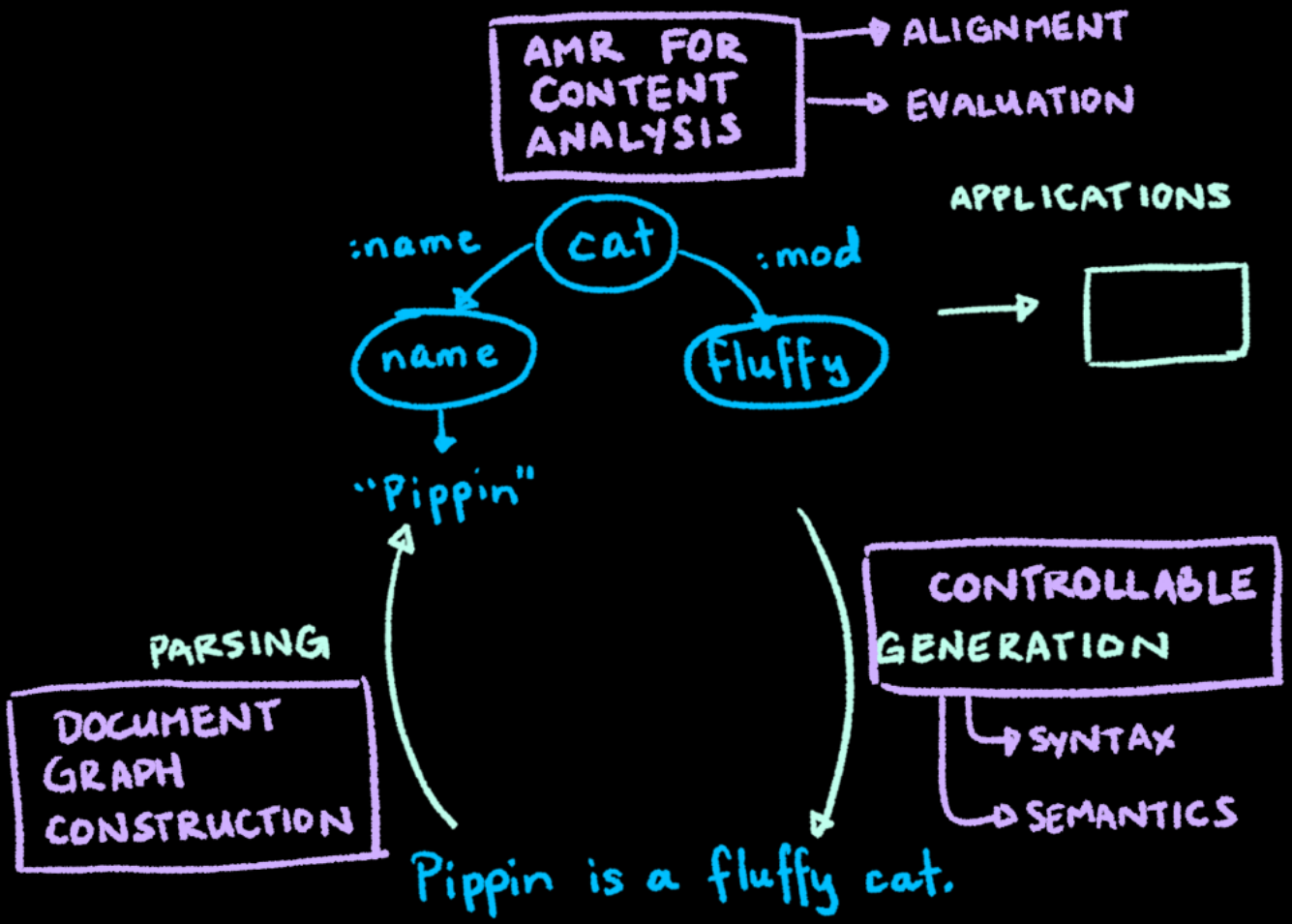
Conclusion

- Content plans give us finer-grained control than available from other methods
- We can get fine-grained control from our fine-grained content plans
- This works even when the label distribution differs from the training set

What problem do we want to address?

Language models are unreliable when it comes to content.

What did we do about it?



What did we do about it?

- We expanded AMR context from sentence to document level;
- We introduced new ways of using AMR to analyze generated text;
- We found new ways to use AMR to generate more subtle and nuanced text.

Contributions

In addressing limitations on context, we...

1. Created a novel method of AMR graph merging
2. Conducted the first evaluation of node merging on downstream content selection performance
3. Collected the first hand-annotated dataset of inter-sentence document-summary node alignments

...lay the foundations for future work on AMR in longer-context settings.

Contributions

In using AMR for content analysis, we...

1. Create a set of novel approaches to node-level AMR alignment with enormous improvements ($\approx 2\times$) over baseline
2. Propose a novel family of AMR-based text evaluation metrics for semantic evaluation

...demonstrate that AMR can be used for explicit, interpretable content analysis.

Contributions

In using AMR as a content plan for generation, we...

1. Propose a novel methodology for using AMR as a content plan for fine-grained (syntactic and semantic) control
2. Demonstrate that AMR yields reliable and consistent control at the concept level

...show that structured content plans give us new and different ways to interact with content not available with plain language models.

Does this work give us anything that LLMs don't?

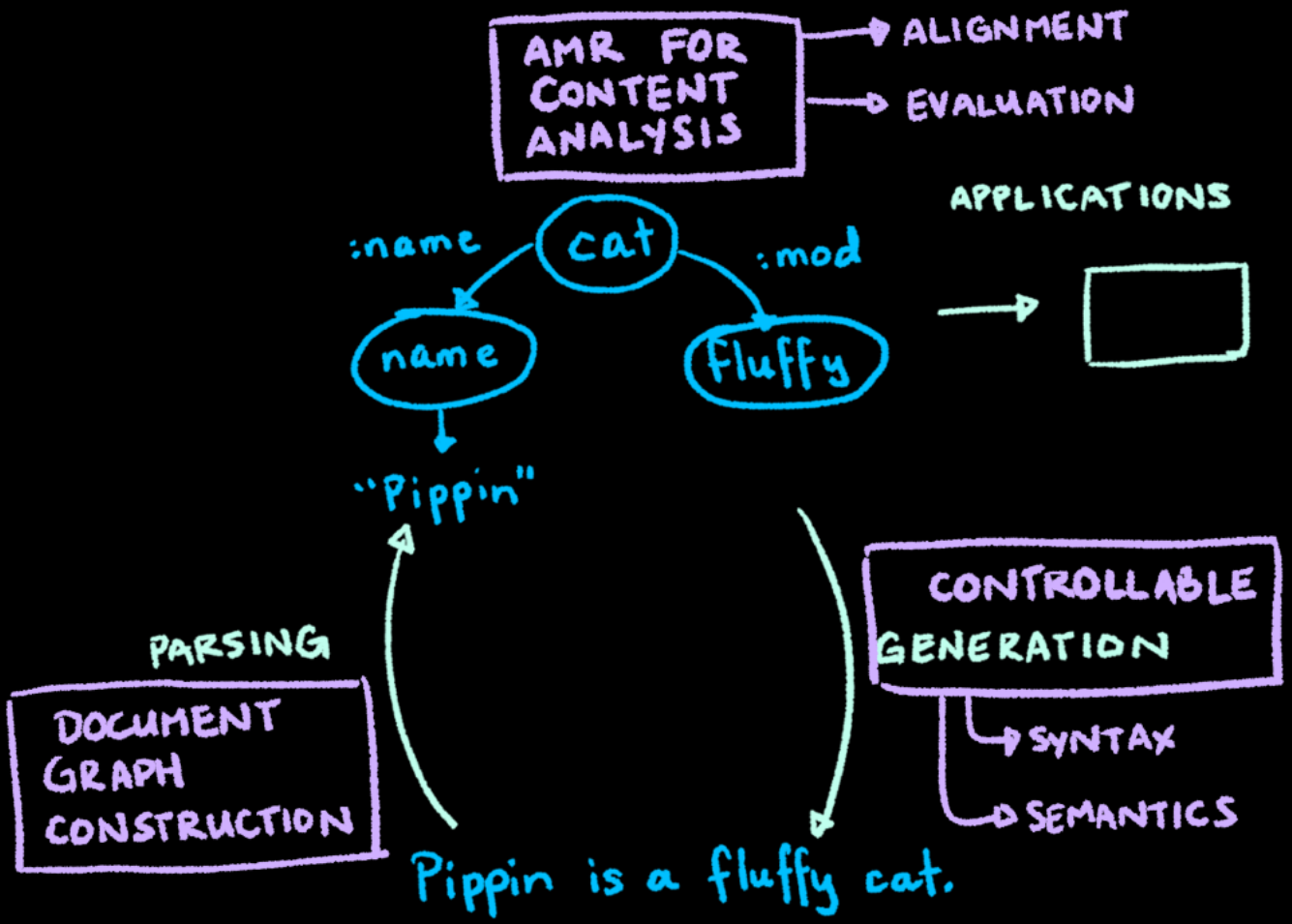
Yes!

- We don't outperform LLMs; we provide different capabilities
- AMR is a complement to LLMs, making their interaction with content more interpretable
- This allows us to interact with content in a more principled manner

Where to from here?

- Controllable models trained directly on structured representations
- More nuanced evaluation metrics over a broader range of graphs and graph metrics
- Referencing content against external knowledge sources

Thanks! :)


AMR FOR CONTENT ANALYSIS
ALIGNMENT
EVALUATION
APPLICATIONS
:name
cat
:mod
name
Fluffy
"Pippin"
PARSING
DOCUMENT GRAPH CONSTRUCTION
CONTROLLABLE GENERATION
SYNTAX
SEMANTICS
Pippin is a fluffy cat.

Talk contents

Experimental results